



Padrão Observer – Peru

Refatoração do Sistema FoodDelivery

Autores:

Franklin Ferreira
Giovanna Rodrigues
Kailla Costa
Lucas Barreto
Lucas Oliveira

Feira de Santana, 2025

Roteiro da Apresentação

Demonstrar o desenvolvimento e a refatoração do sistema de pedidos do FoodDelivery, com foco na implementação do padrão de projeto Observer em Java.

1. Introdução: Conceito do projeto e o desafio da comunicação

2. Solução Proposta:
Introdução ao Padrão de Projeto Observer

3. Diagrama de Classes:
Arquitetura final, ilustrando a relação entre o Subject (Pedido) e os Observers (Cliente, FilaEntrega)

4. Funcionalidades e Regras:
Funções implementadas e regras de associação

5. Código em Execução:
Imagem de evidência do MenuCLI

6. Vantagens da Utilização:
Por que usar o Observer?

7. Referências: Fontes utilizadas para construção do projeto

Introdução

O desafio da Comunicação no FoodDelivery

Cenário: O **Pedido** muda de estado constantemente (AGUARDANDO, PREPARANDO, etc.).

Necessidade: Vários componentes precisam reagir a essas mudanças:

- O *Cliente* quer ser notificado do seu pedido em tempo real
- A *Fila de Entrega* precisa saber quando um pedido está pronto

Problema Central: Acoplamento e Responsabilidade Excessiva



Introdução



O desafio da Comunicação no FoodDelivery

O **Pedido** age como um chefe que avisa cada funcionário (componente) individualmente, em vez de usar um quadro de avisos.

O chefe fica sobrecarregado: Ele precisa conhecer cada funcionário (Cliente, FilaDeEntrega).

Frágil: Se um funcionário muda de lugar, a comunicação falha.

Não escalável: A cada novo funcionário, o trabalho do chefe aumenta.



Solução Proposta

O Padrão Observer

Permite que um objeto notifique múltiplos "observadores" sobre mudanças de estado, sem precisar conhecê-los diretamente

Define uma relação de **um-para-muitos** entre objetos

Pedido (Publicador): Apenas publica suas atualizações de status. Ele não conhece seus assinantes.

Cliente e FilaDeEntrega (Assinantes): Assinam as atualizações do Pedido e reagem de forma autônoma.

Resultado: O Pedido não controla mais ninguém, apenas informa.

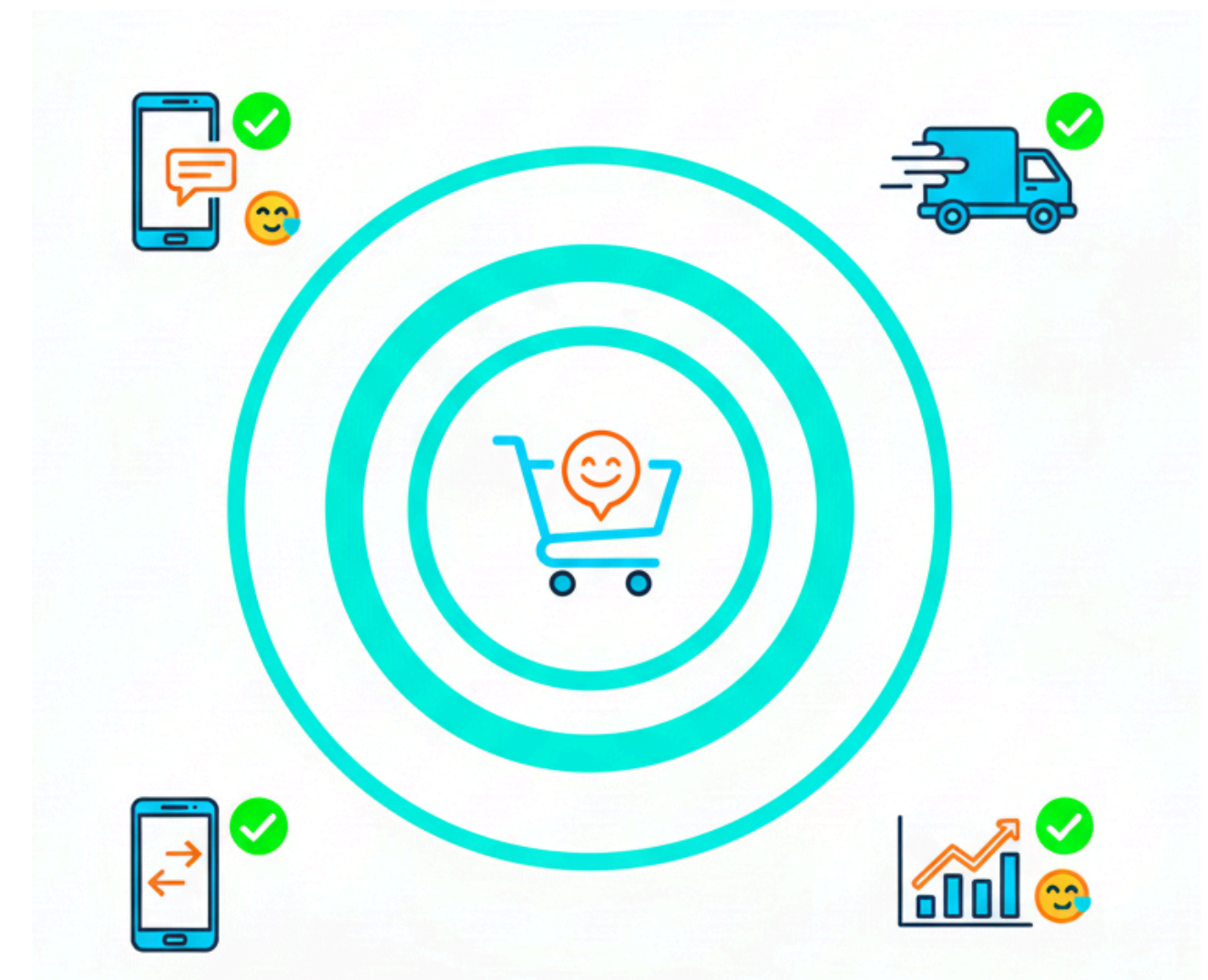
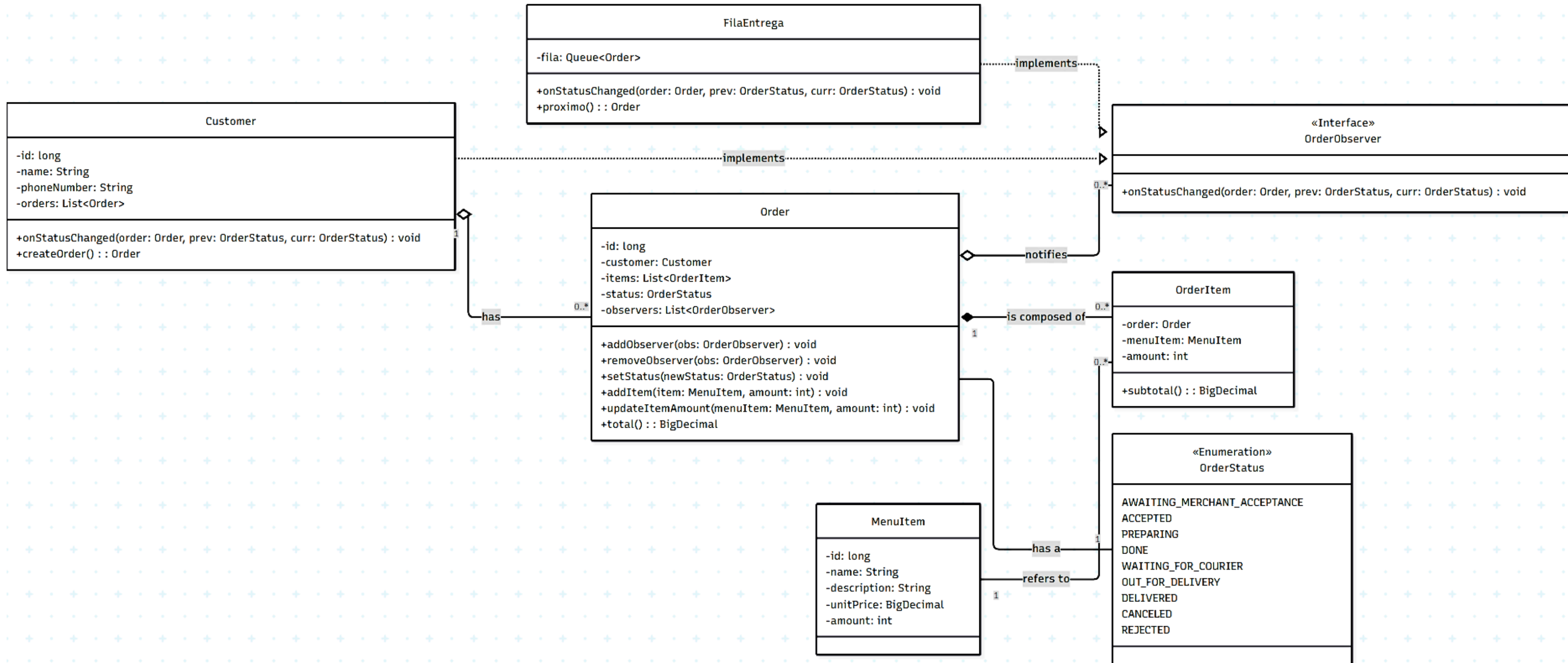


Diagrama de Classes



Funcionalidades



Inscrição (addObserver):

Permite que um componente (como Cliente ou FilaDeEntrega) comece a "seguir" as atualizações de um Pedido.

Notificação (notifyObservers):

Quando o Pedido muda de estado, ele dispara um sinal para todos os componentes inscritos em sua lista.

Reação (update): É o método que cada "ouvinte" implementa para reagir ao sinal.

- O Cliente usa o update para exibir uma mensagem na tela.
- A FilaDeEntrega usa o update para adicionar o pedido a uma fila quando o status for "Aguardando Entregador".

Cancelamento (removeObserver):

Permite que qualquer componente deixe de seguir as atualizações de forma dinâmica.

MenuCLI



Quando executamos o programa:

```
=== TiaLu Delivery ===  
1 - Aceitar pedido  
2 - Preparar pedido  
3 - Aguardar entregador  
4 - Sair para entrega  
5 - Entregue  
6 - Rejeitar  
7 - Cancelar  
0 - Sair  
Escolha uma opção: 1  
[Cliente] Lucas: seu pedido #1 foi aceito.
```


Vantagens

Por que usar o Observer?



-  **Baixo Acoplamento:** O Pedido não precisa "conhecer" o Cliente.
-  **Extensibilidade:** Fácil adicionar novos "ouvintes" (Observers) sem alterar o Pedido.
-  **Coesão:** Responsabilidades ficam bem separadas e organizadas.
-  **Resultado:** Código mais limpo, flexível e fácil de manter.

Referências



BAELDUNG. Java Observer Pattern. Baeldung. Disponível em: <https://www.baeldung.com/java-observer-pattern>. Acesso em: 26 ago. 2025.

SANTOS, Nelio. Padrão de Projeto Observer em Java. DevMedia. Disponível em: <https://www.devmedia.com.br/padrao-de-projeto-observer-em-java/26163> Acesso em: 29 ago. 2025.